

Bahria University, Islamabad

Department of Software Engineering

Web Engineering Lab (Fall-2025)

Instructor: Engr. Subas Bilal

Student: Waleed Ahmed

Enrollment: 01-131232-093

Lab Journal: 09

Date: 4/12/2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1					
2					
3					
4					
5					

Comments:

Signature

Lab Task: Set up Node.js and implement backend authentication (user login and registration) Backend

Lab Objectives

By the end of this lab, students will be able to: - Set up a Node.js backend using Express - Connect a Node.js application to MongoDB Atlas using Mongoose - Implement user authentication (Signup & Login) - Use bcrypt for password hashing - Use JSON Web Tokens (JWT) for authentication - Test API endpoints using Postman

---

Software / Tools Required

- Node.js (installed)
- Visual Studio Code
- MongoDB Atlas account
- Postman

Lab Description

In this lab, you will create the backend of a MERN application focused on user authentication. The backend will support user signup and login using secure password hashing and JWT-based authentication, with data stored in MongoDB Atlas.

Step 1: Project Setup

1. Create a new folder named:

   `NEW MERN`

2. Open the folder in **Visual Studio Code**.

3. Inside the project folder, create another folder named:

   `backend`

4. Open the terminal in VS Code and navigate to the `backend` folder.

Ans Done

Step 2: Initialize Node Project

Run the following commands in the terminal:

```
npm init -y
npm install express mongoose dotenv jsonwebtoken bcryptjs
npm install nodemon -D
```

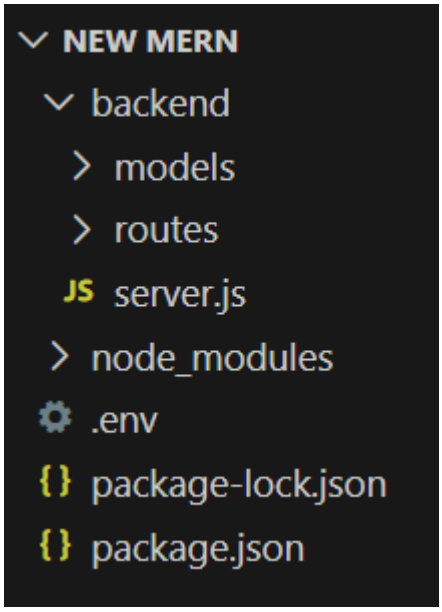
These packages are used for: - **express** – web framework - **mongoose** – MongoDB object modeling - **dotenv** – environment variables - **jsonwebtoken** – authentication tokens - **bcryptjs** – password hashing - **nodemon (-D)** – development tool to auto-restart the server on file changes

Ans. Done

Step 3: Folder Structure

Create the following folder structure inside `NEW MERN folder in VS CODE`:

```
backend
├── models
│   └── User.js
├── routes
│   └── auth.js
├── .env
├── server.js
└── package.json
```

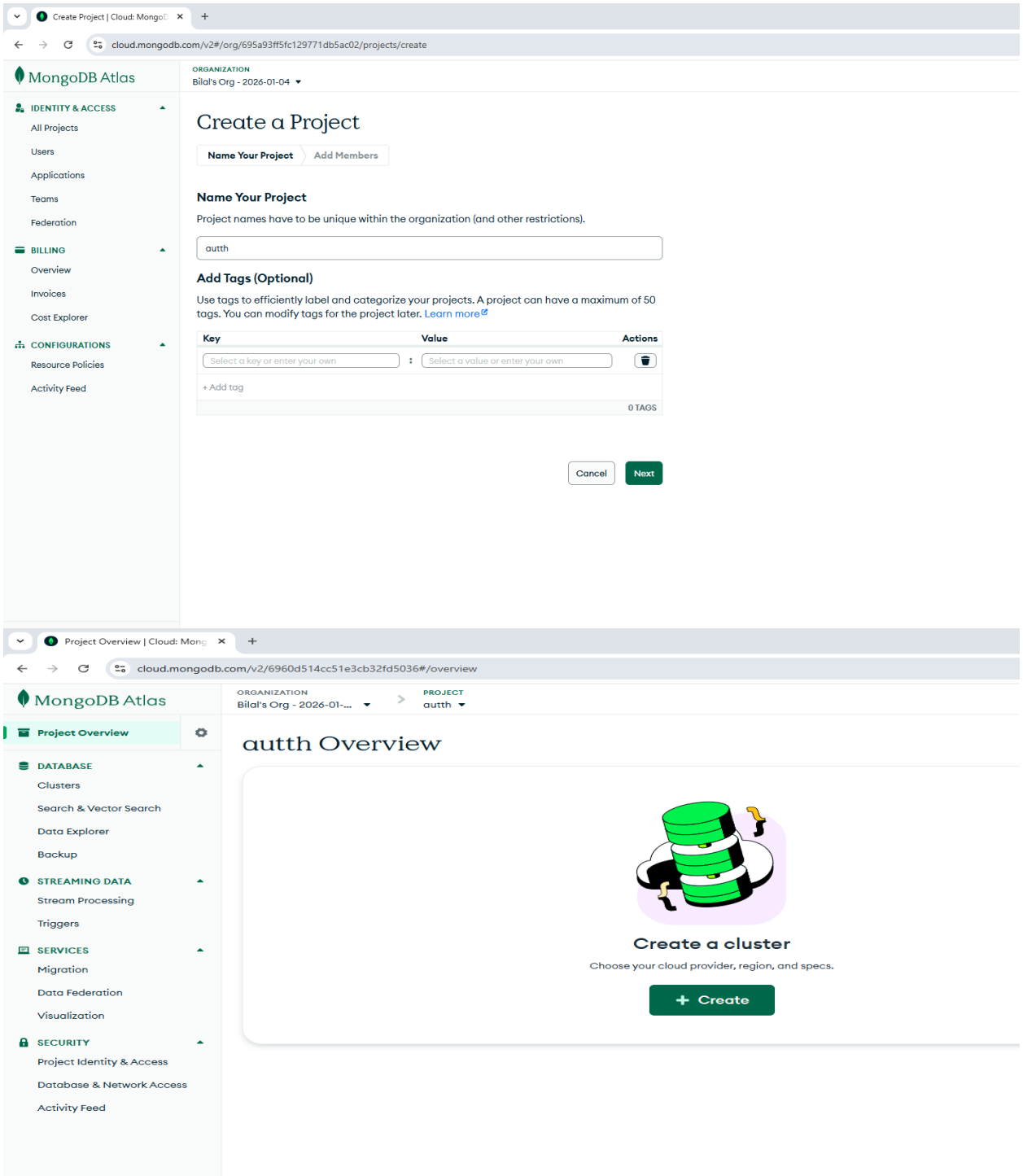


`Ans. Done`

Step 4: MongoDB Atlas Setup

1. Create an account on **MongoDB Atlas**.
2. Create a new **Cluster**.
3. Create a database user (username & password).

4. Whitelist your IP address.
5. Copy the connection string.



Create a cluster

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

M10 — $0.08/hour

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

Flex — From $0.011/hour, Up to $30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

Free

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

Free forever! Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name
You cannot change the name once the cluster is created.

Cluster0

Provider

aws | Google Cloud | Azure

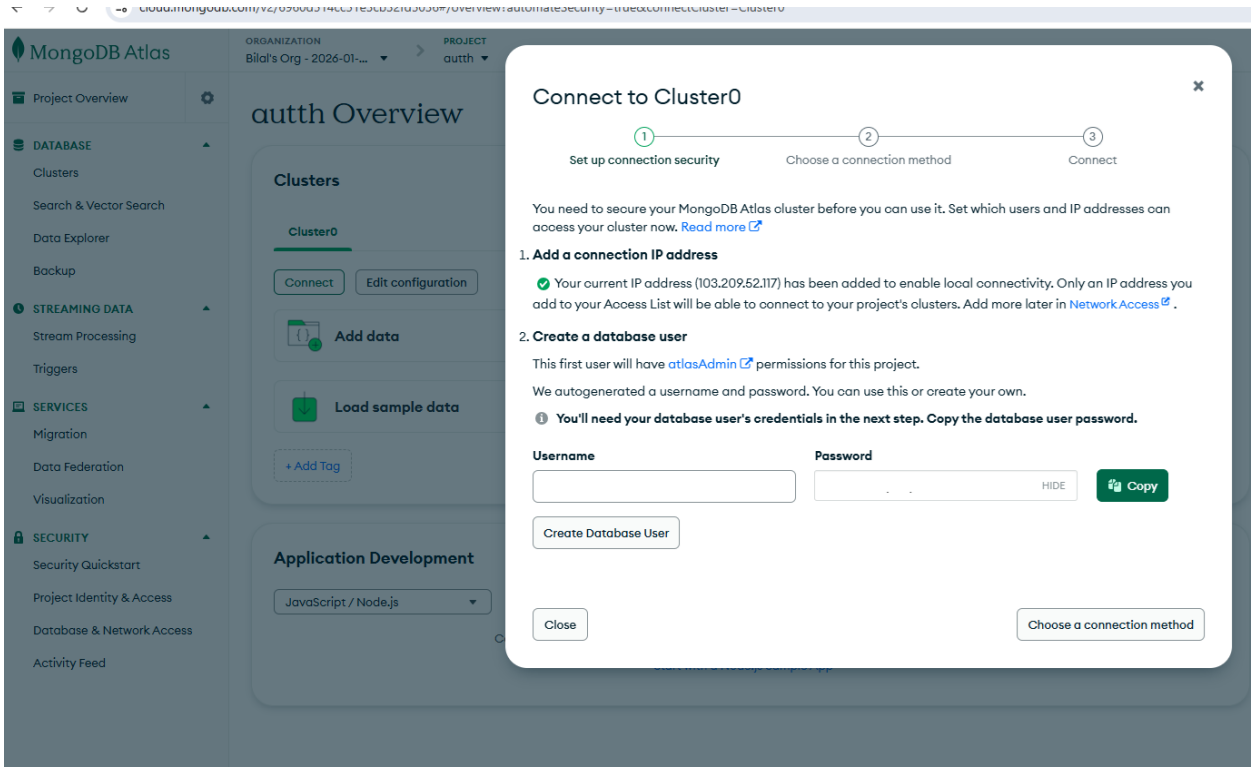
Region

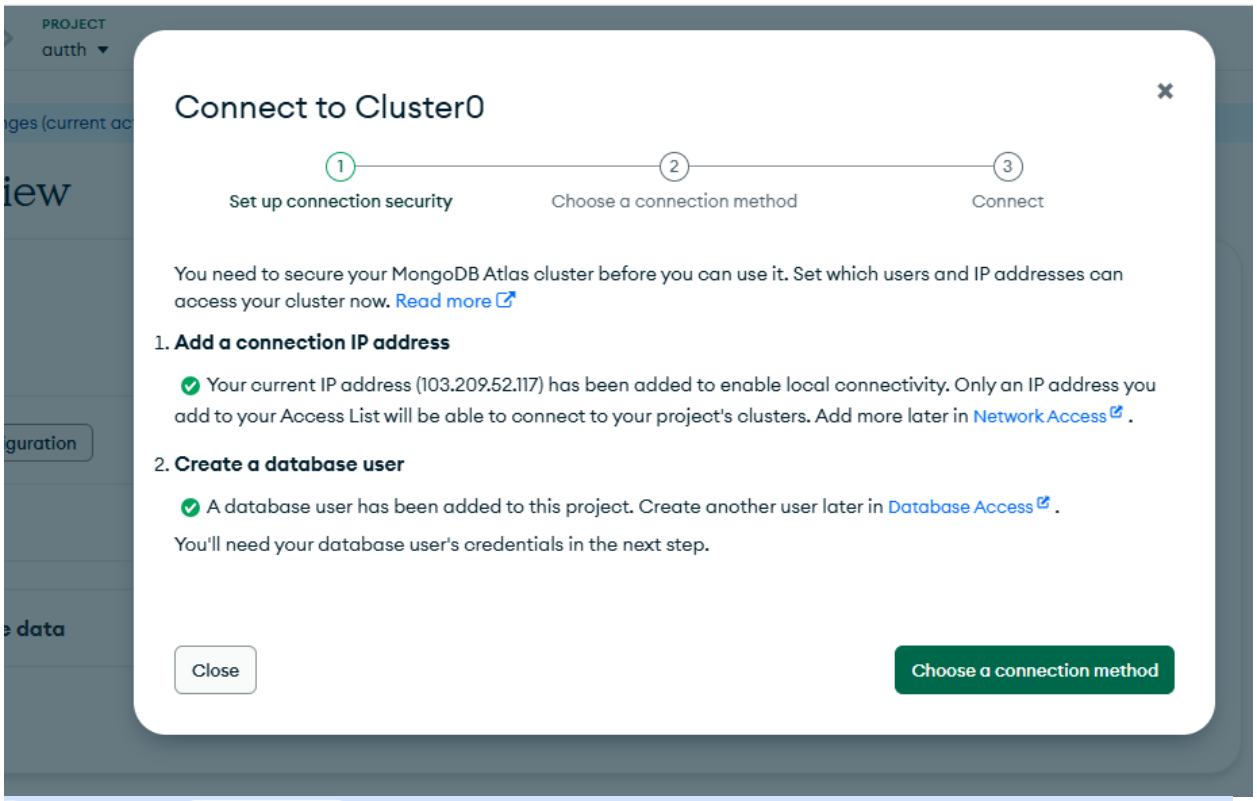
Mumbai (ap-south-1)

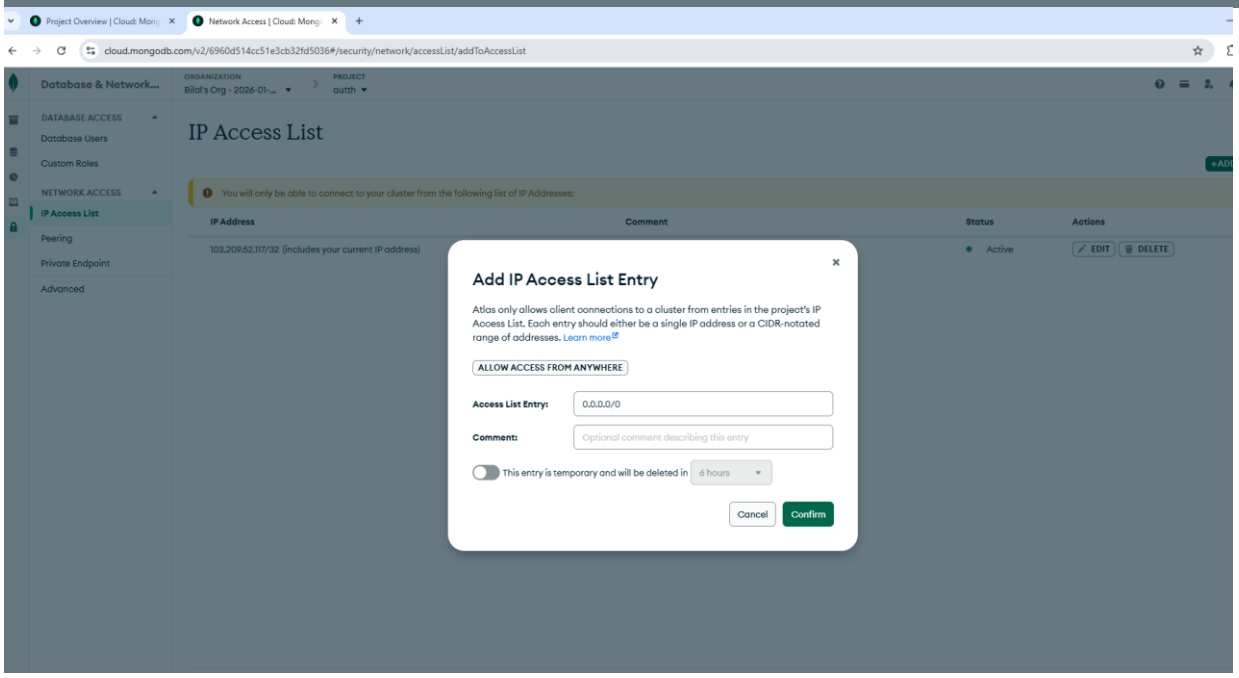
Quick setup

- [x] Automate security setup
- [ ] Preload sample dataset

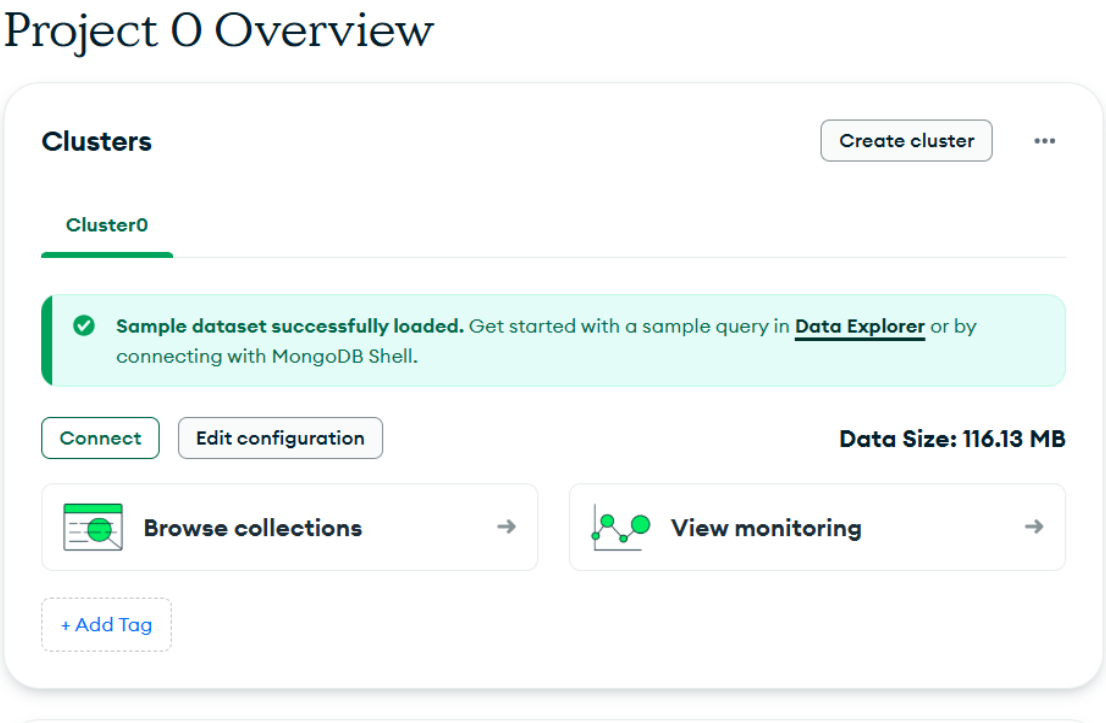
I'll do this later | Go to Advanced Configuration | Create Deployment




PROJECT
autth
Connect to Cluster0
1
Set up connection security
2
Choose a connection method
3
Connect
You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. Read more
1. Add a connection IP address
Your current IP address (103.209.52.117) has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's clusters. Add more later in Network Access .
2. Create a database user
A database user has been added to this project. Create another user later in Database Access .
You'll need your database user's credentials in the next step.
Close
Choose a connection method


IP Access List
Add IP Access List Entry
ALLOW ACCESS FROM ANYWHERE
Access List Entry:
0.0.0.0/0
Comment:
Cancel
Confirm

Ans. Done



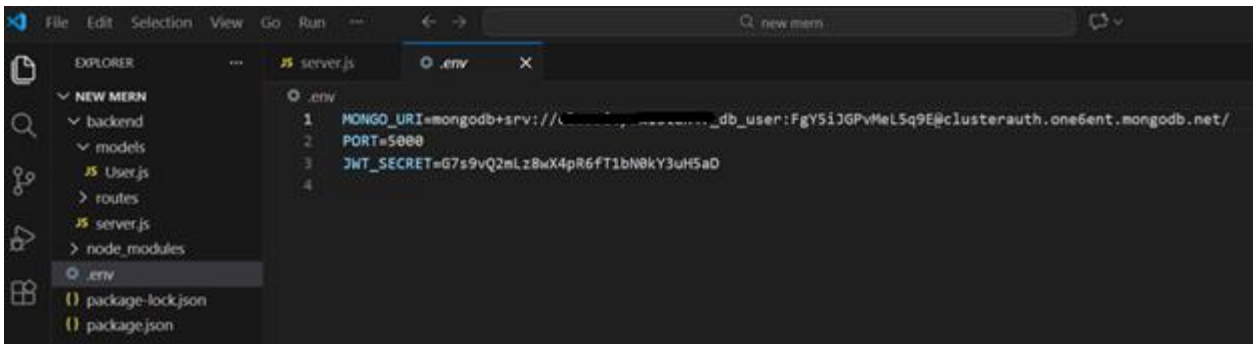
Step 5: Environment Configuration (.env)

Create a `.env` file inside the backend folder and add the following:

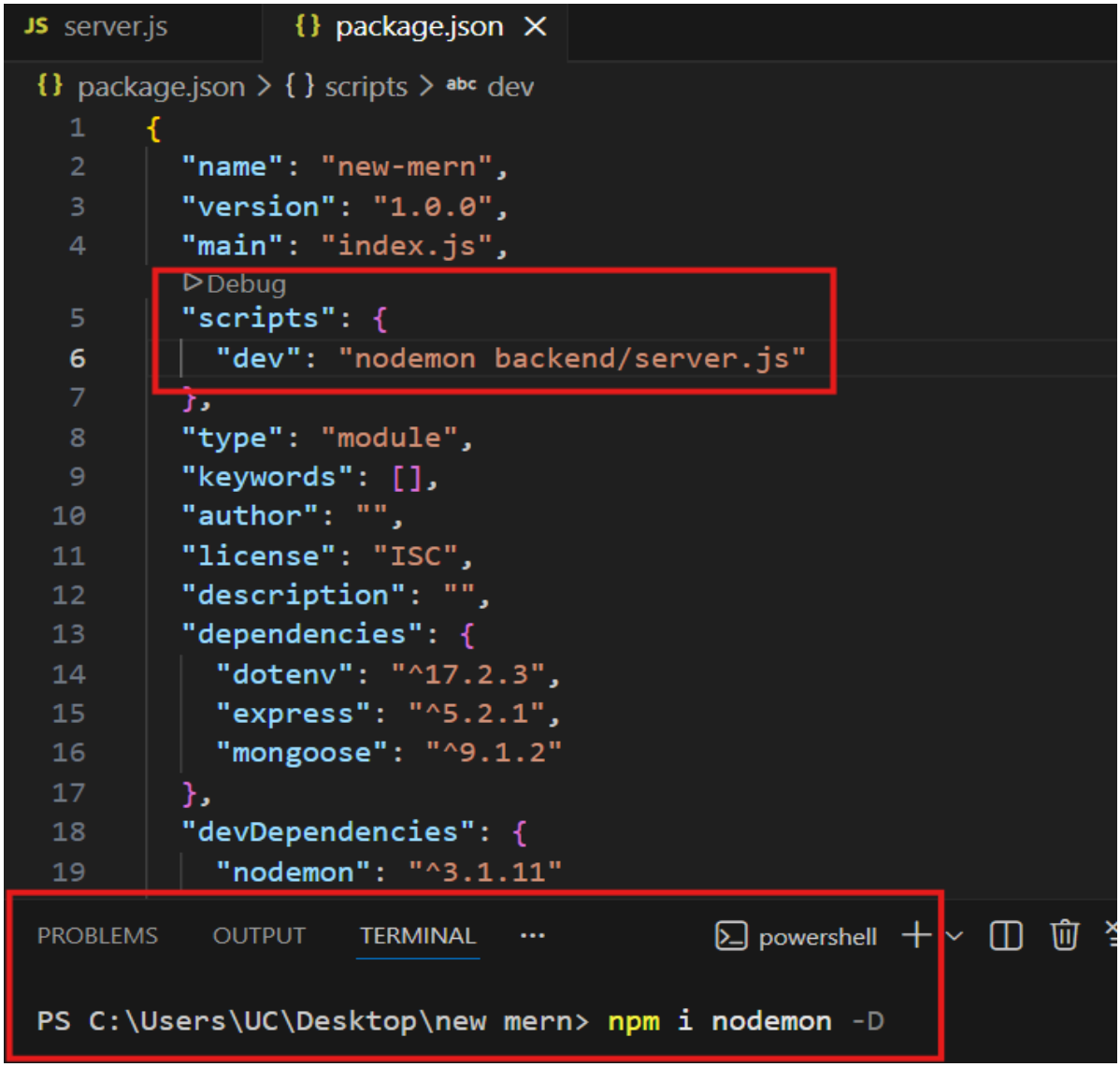
```
MONGO_URI=mongodb+srv://<username>:<password>@clusterauth.one6ent.mo
ngodb.net/
PORT=5000
JWT_SECRET=G7s9vQ2mLz8wX4pR6fT1bN0kY3uH5aD
```

Replace `<username>` and `<password>` with your MongoDB Atlas credentials.

Ans. Done



Step 6: Package.JSON



Step 7: Server Setup (server.js)

The `server.js` file acts as the entry point of the backend application. It: - Initializes Express - Connects to MongoDB - Uses authentication routes - Starts the server

```
import express from 'express';
import mongoose from 'mongoose';
import dotenv from 'dotenv';
import dns from 'node:dns';
import cors from 'cors';
import authRoutes from './routes/auth.js';

dns.setServers(['8.8.8.8', '1.1.1.1']);
```

```
// Load environment variables
dotenv.config();
const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Request logging middleware
app.use((req, res, next) => {
  console.log('Incoming request:', req.method, req.url);
  console.log('Request body:', req.body);
  next();
});

// Routes
app.use('/api/auth', authRoutes);

// Root route
app.get('/', (req, res) => {
  res.json({ message: 'Authentication API is running' });
});

// MongoDB connection
mongoose.connect(process.env.MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log('Database:', mongoose.connection.db.databaseName);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
    process.exit(1);
  });

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
  console.log(`http://localhost:${PORT}`);
});
```

Step 7: User Model (models/User.js)

The User model defines the structure of user data stored in MongoDB, including:
- Username - Email - Password – Timestamps

```
import mongoose from 'mongoose';
```

```
const userSchema = new mongoose.Schema(
  {
    username: {
      type: String,
      required: [true, 'Username is required'],
      unique: true,
      trim: true,
      minlength: [3, 'Username must be at least 3 characters long']
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email']
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: [6, 'Password must be at least 6 characters long']
    },
  },
  { timestamps: true }
);

export default mongoose.model('User', userSchema);
```

Step 8: Authentication Routes (routes/auth.js)

The authentication routes provide:

Signup API

- Checks if user already exists
- Hashes password using bcrypt
- Saves user to MongoDB

Login API

- Verifies email and password
- Generates JWT token
- Returns user data and token

```
import express from 'express';
import bcrypt from 'bcryptjs';
```

```
import jwt from 'jsonwebtoken';
import User from '../models/User.js';

const router = express.Router();

router.post('/signup', async (req, res) => {
  try {
    const { username, email, password } = req.body;

    if (!username || !email || !password) {
      return res.status(400).json({
        success: false,
        message: 'Please provide all required fields: username, email,
password'
      });
    }

    const existingUser = await User.findOne({
      $or: [{ email }, { username }]
    });

    if (existingUser) {
      const field = existingUser.email === email ? 'Email' :
'Username';
      return res.status(400).json({
        success: false,
        message: `${field} already exists`
      });
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(password, salt);
    const newUser = new User({
      username,
      email: email.toLowerCase(),
      password: hashedPassword
    });

    await newUser.save();

    const token = jwt.sign(
      { id: newUser._id, email: newUser.email },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.status(201).json({
      success: true,
```

```
      message: 'User created successfully',
      token,
      user: {
        id: newUser._id,
        username: newUser.username,
        email: newUser.email
      }
    });

  } catch (err) {
    console.error('Signup error:', err);
    res.status(500).json({
      success: false,
      message: 'Server error. Please try again later.'
    });
  }
});

router.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        success: false,
        message: 'Please provide both email and password'
      });
    }

    const user = await User.findOne({ email: email.toLowerCase() });

    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Invalid credentials'
      });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if (!isMatch) {
      return res.status(401).json({
        success: false,
        message: 'Invalid credentials'
      });
    }

    const token = jwt.sign(
```

```
      { id: user._id, email: user.email },
      process.env.JWT_SECRET,
      { expiresIn: '7d' }
    );

    res.json({
      success: true,
      message: 'Login successful',
      token,
      user: {
        id: user._id,
        username: user.username,
        email: user.email
      }
    });

  } catch (err) {
    console.error('Login error:', err);
    res.status(500).json({
      success: false,
      message: 'Server error. Please try again later.'
    });
  }
});

router.get('/me', async (req, res) => {
  try {
    const token = req.headers.authorization?.split(' ')[1];

    if (!token) {
      return res.status(401).json({ message: 'No token provided' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');

    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    res.json({ user });
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
});

export default router;
```

Step 9: Running the Server

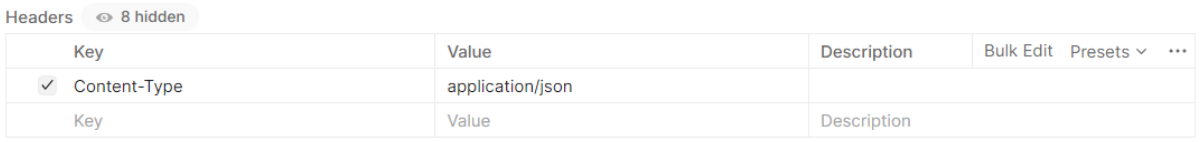
Start the backend server using:

```
npm run dev
```

Expected output:

```
MongoDB connected
Server running
```

Step 10: Testing with Postman



Sign-in:

Body:

```
{
  "username": "waleed85147",
  "email": "waleed85147@gmail.com",
  "password": "12355"
}
```

Output:

```
{
  "message": "User created successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "67f8a1b2c3d4e5f6a7b8c9d0",
    "username": "waleed85147",
    "email": "waleed85147@gmaik.com"
  }
}
```

Login:

Body:

```
{
  "email": "waleed85147@gmail.com",
  "password": "12355"
}
```

Output:

```
{
  "success": true,
  "message": "Login successful",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5ZGJjOTNiMTY0Y2NkMDllN2EwY2YwYiIsI
```

mVtYWlsIjoid2FsZWVkODUxNDdAZ21haWwuY29tIiwiaWF0IjoxNzc2MDEyMDY3LCJleHAiOjE
3NzY2MTY4Njd9.IzpttGmAqhJqjgQVC0zh07dE0G7iLK8-3TMlFqSkkuo",
  "user": {
    "id": "69dbc93b164ccd09e7a0cf0b",
    "username": "waleed85147",
    "email": "waleed85147@gmail.com"
  }
}

Sign-in (Error Case):

{
  "success": false,
  "message": "Email already exists"
}

Login (Error Case): From wrong password

{
  "success": **false**,
  "message": "Invalid credentials"
}

Step 11: MongoDB Verification

- Open MongoDB Atlas
- Navigate to **Collections**
- Verify that the user data is stored in the database

Ans. Done


```
_id: ObjectId('69dbc93b164ccd09e7a0cf0b')
username : "waleed85147"
email : "waleed85147@gmail.com"
password : "$2a$10$/7kfLS5TnMa087nuQHal/.MmiucXCLmraywnsgwO0aE9YWiGS24de"
createdAt : 2026-04-12T16:32:59.528+00:00
updatedAt : 2026-04-12T16:32:59.528+00:00
__v : 0
```

Expected Learning Outcomes

Students will be able to: - Build a secure backend using Node.js and Express - Integrate MongoDB Atlas with Mongoose - Implement authentication using JWT - Test REST APIs using Postman